```
function advancePhase() public onlyBeneficiary {                ❸
    // If already in Done phase, reset to Init phase
    if (currentPhase == Phase.Done) {
        currentPhase = Phase.Init;
    } else {
    // Else, increment the phase
    // Conversion to uint needed as enums are internally uints
    uint nextPhase = uint(currentPhase) + 1;
    currentPhase = Phase(nextPhase);
    }

    if (currentPhase == Phase.Reveal) emit RevealStarted();     ❹
    if (currentPhase == Phase.Bidding) emit BiddingStarted();   ❹
}

function bid(bytes32 blindBid) public payable validPhase(Phase.Bidding)
     { ... }

function reveal(uint value, bytes32 secret) public
                      validPhase(Phase.Reveal) { ... }


function placeBid(address bidder, uint value) internal returns
                                              (bool success) {  ❺
    ... }

function withdraw() public {
    ... }

function auctionEnd() public validPhase(Phase.Done) {
    beneficiary.transfer(highestBid);
    emit AuctionEnded(highestBidder, highestBid);               ❻
}}
```

❶ Phases will be set only by the beneficiary (auctioneer).

❷ Definition of events

❸ Phases will be set only by the beneficiary (auctioneer).

❹ Emit appropriate event

❺ Internal function can be called only from the contract itself

❻ Send the highest bid to the beneficiary; announce the end of the auction.

ADVANCEPHASE FUNCTION

A significant addition to the blind auction code is the function `advancePhase()`, which replaces a version of `changeState()` function that appeared in chapter 5. In this